

数据回归

1. 实验目的

通过神经网络完成数据回归问题；

2. 实验数据

具有一定随机性的散点分布数据。

3. 实验环境

SUB KIT NX 嵌入式开发板。

4. 准备工作

4.1 导入必要的模块

实验用到的模块：

- numpy 模块，数据处理
- torch 模块，构造网络
- matplotlib 模块，可视化显示结果

导入模块：

```
import numpy as np          #导入 numpy
import torch                #导入 torch
import torch.nn as nn       #导入 torch.nn，简写为 nn
import matplotlib.pyplot as plt #导入 matplotlib.pyplot，简写为 plt
```

4.2 构造样本数据

给出一个数组 x ，然后基于一个正弦函数，加上噪声数据得到一组待回归的训练数据样本 y 。

```
x = torch.unsqueeze(torch.linspace(-np.pi, np.pi*2, 200), dim=1)
```

```
y = torch.cos(x) + 0.3 * torch.rand(x.size()) # 添加随机数
```

- x 数据

`torch.linspace` 生成连续间隔的数据（等差数列），第一个参数表示起点，第二个参数表示终点，第三个参数表示将这个区间分成平均多少份，即生成多少数

据；

由于 torch 只能处理二维数据，所以用 torch.unsqueeze 给数据添加一个维度，dim 表示添加在第几维，这样生成的等差数列转换成二维的数据可以输入网络；

torch.rand 返回[0,1)之间的均匀分布。

- y 数据

采用 torch.sin()和 torch.rand()接口构建一个在 sin()函数周围的一些随机散点。
只有转换成二维的数据才能输入网络。

- 一维数据转换成二维数据

一维数组：

```
x = torch.linspace(- np.pi, np.pi, 3) #构建等差数列
```

```
print(x)
```

输出结果如下：

```
tensor([-3.1416, 0.0000, 3.1416])
```

二维数组：

```
x = torch.unsqueeze(torch.linspace(- np.pi, np.pi, 3), dim=1) #构建等差数列并转换成二维数组
```

```
print(x)
```

输出结果：

```
tensor([[ -3.1416],\n [ 0.0000],\n [ 3.1416]]) # \n 是回车换行
```

数组中多添加了中括号，增加了维度，数据可以输入到网络中去。

5. 构建网络

所构建网络隐层节点数为 20，网络为三层结构，激活函数为 ReLU，网络结构如图 2.1 所示。

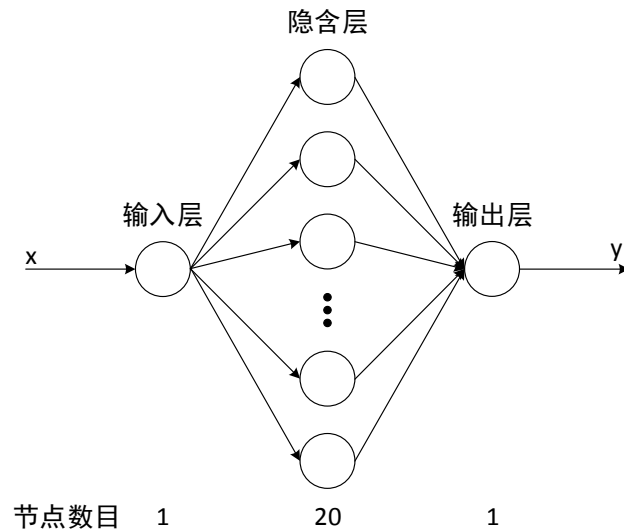


图 2.1 网络结构

用类的方式构建网络（见补充说明）：

```
class Net(nn.Module):                #定义类

    def __init__(self):              #初始化函数定义神经网络的基本架构
        #使用 Net 的父类的初始化方法（运行 nn.Module 的初始化函数）
        super(Net, self).__init__()

        #Sequential 是一个有序的容器，神经网络模块将按照传入构造器的顺序
        #依次被添加到计算图中执行

        self.predict = nn.Sequential(
            nn.Linear(1, 20),
            nn.ReLU(),
            nn.Linear(20, 1)
        )

    def forward(self, x):              #定义前向传播过程
        out = self.predict(x)         #将 x 传入网络
        return out                    #返回预测值
```

说明：

- 神经网络的类一般都继承 `torch.nn.Module`；
- `__init__()`和 `forward()`两个函数是自定义类的主要函数。在`__init__()`中要添加一句 `super(Net, self).__init__()`，这是固定的标准写法，用于继承父类的初始化

函数。__init__()只是对神经网络的模块进行了声明，真正的搭建是在 forward() 中实现；

- 自定义类中的成员都通过 self 指针来进行访问，所以参数列表中都包含了 self。

5.1 训练网络

网络训练采用 SGD 算法进行优化，采用 MSE 作为损失函数，训练过程通过 for 循环实现：

```
net = Net()
optimizer = torch.optim.SGD(net.parameters(), lr=0.05)
loss_func = nn.MSELoss()
for epoch in range(6000):          # 训练部分
    out = net(x)
    loss = loss_func(out, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

5.2 动态显示

为了能够动态显示训练过程，需要在该循环之前通过 plt.ion()打开交互模式，然后用 if 语句设置每隔多少 epoch 显示一次当前训练的结果，利用 plt.scatter() 和 plt.plot()分别绘制样本散点图和拟合曲线，最后使用 plt.ioff()关闭交互模式，使用 plt.show()定格显示最后一次训练结果。

```
plt.ion()
for epoch in range(Num):
    #网络优化
    if epoch % 显示频率 == 0:
        plt.cla()
        plt.scatter(***)
        plt.plot(***)
        plt.text(***)
        plt.pause(***)
```

```
plt.show()
```

```
plt.ioff()
```

```
plt.show()
```

6. 实验任务

同学自己编写完整的数据回归的训练程序和测试程序，并显示输出结果。同学可以尝试不同的层数、隐层节点数、改进或自行设计网络模型，完成实验任务。